

Flourish Visualization Pipeline using R

Flourish for the design · R for the data

Proposal at a glance Build and style the visualization once in Flourish, then use R to supply fresh SPI data whenever the index is updated.

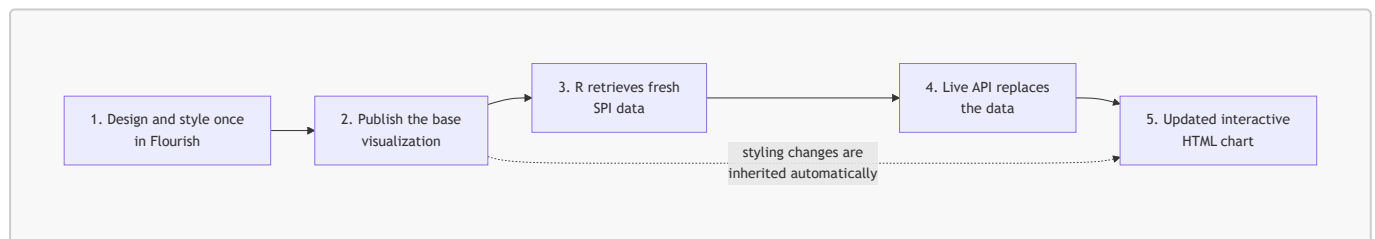
Idea: Adopt a *Flourish-as-template* + *R-as-data* pipeline for SPI charts, instead of building and maintaining custom visualization functions in R and publishing their output to our website.

Context

We need recurring, publication-quality SPI visualizations (e.g. the choropleth SPI map) that refresh as new **spiR** data lands. Two ways to get there:

- **Option A — Build it all in R.** Write and maintain R plotting functions (ggplot/leaflet/plotly), export static images or HTML widgets, and publish those to our web page.
- **Option B — Flourish templates driven by R data (recommended).** Design the chart once in Flourish (styling, interactivity, branding), and use R only to feed it fresh data through Flourish's Live API.

How the pipeline works

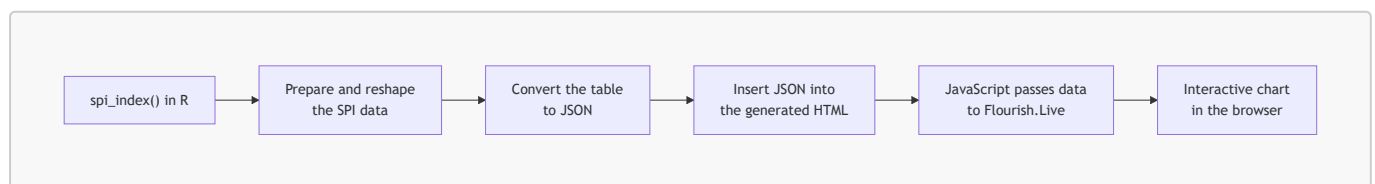


- Flourish holds the **look** (colors, projection, legend, tooltips, title, geometry).
- R holds the **data** (`spi_index()` → the values to display).
- Each run re-reads the base styling, so any design change made in Flourish is inherited automatically — no code change required.

This is Flourish's officially documented "replicate a published visualization" pattern.

How the data reaches the chart

R and the web browser use different environments. R prepares the SPI data, but the browser runs HTML and JavaScript. **JSON is the bridge between them.**



For example, R starts with a table:

date	Caribbean	North America
2016	42.08	87.12
2017	44.10	86.78

`jsonlite::toJSON()` converts it to a format that JavaScript can read:

```
[
  {"date": 2016, "Caribbean": 42.08, "North America": 87.12},
  {"date": 2017, "Caribbean": 44.10, "North America": 86.78}
]
```

The generated HTML stores that JSON in a JavaScript variable and sends it to the Flourish Live API. **JSON is not the visualization and does not contain the design**; it only carries the data. Flourish supplies the template, colors, labels, tooltips and interactivity.

In this pipeline, the JSON is embedded in the generated HTML, so the data is fixed at the moment the R script runs. To show newer SPI values, re-run R and replace the HTML file on the website. The browser still needs access to the Flourish JavaScript library and Live API.

One pipeline, two implementation paths

Both the regional trend chart and the map use the same principle: **Flourish stores the design, and R supplies the data**. The implementation is different because the chart types have different data structures.

	Regional trend chart	Projection map
Example script	<code>flourish_spi_trend.R</code>	<code>flourish_map.R</code>
R method	<code>flourishcharts</code>	Direct Flourish Live API
Data binding	<code>bind_line_bar_pie_data()</code>	No equivalent map binding is available in <code>flourishcharts</code>
Main data	One table: <code>date</code> + one column per region	Several datasets, including <code>regions_geometry</code>
JSON / JavaScript	Created internally by <code>flourishcharts</code>	Written explicitly in the script
Special requirement	Match the line-chart bindings	Preserve the country geometry while replacing <code>regions</code>

Regional trend: `flourishcharts`

For line, bar and pie charts, the package provides the intended R workflow:

```
chart <- flourish(
  base_visualisation_id = 28718501,
  api_key = Sys.getenv("FLOURISH_API_KEY")
) |>
  bind_line_bar_pie_data(
    data = chart_data,
    label = "date",
    value = setdiff(names(chart_data), "date")
  )
```

Here, `chart_data` remains an R data frame. `flourishcharts` handles the conversion to the browser-readable format and the HTML widget internally.

Projection map: direct Live API

The map uses a more manual path because its geometry is a separate dataset. `flourish_map.R` therefore:

1. Gets fresh SPI values and names the columns `Economy` and `SPI.INDEX` to match the base bindings.
2. Converts the replacement `regions` table to JSON.
3. Retrieves the complete published base configuration.
4. Keeps `regions_geometry`, styling, bindings and metadata unchanged.
5. Replaces only `data.regions` and passes the complete configuration to `Flourish.Live`.

This is not a different visualization strategy. It is the map-specific implementation of the same base-visualization pattern.

Important: `flourishcharts` is the preferred method for the trend chart. The map should keep using the direct Live API unless Flourish adds a dedicated projection-map binding function.

Step-by-step: reuse and update a Flourish chart from R

This is the concrete recipe implemented by `flourish_map.R`. Complete **Part A once**; after that, repeat **Part B** whenever fresh data is available.

Part A · One-time setup

1. **Build and style the chart in Flourish.** Design it fully in the Flourish editor (colors, projection, legend, tooltips, title, column bindings). This becomes the **styled base** — R never re-creates the look.
2. **Publish it** and copy its **visualization ID** (the number in the published URL, e.g. `26427135`). This is the `base_visualisation_id` R will point to.
3. **Note the exact column names the base is bound to.** For the SPI map the base binds `Economy` (ISO3 join key) and `SPI.INDEX` (the value). R must resupply **these same names**, or the join breaks.
4. **Get an enterprise Live API key** and store it in your `.Renviron` (never hard-code it):

```
# ~/.Renviron (or project-level .Renviron)
FLOURISH_API_KEY=your_enterprise_key_here
```

Restart R so the key loads, then confirm with `Sys.getenv("FLOURISH_API_KEY")`.

5. Install the R packages used by the selected script:

```
install.packages(c("dplyr", "jsonlite", "flourishcharts", "htmlwidgets"))
# spiR: install from your internal source / GitHub
```

The trend chart uses `flourishcharts` and `htmlwidgets`. The map uses `jsonlite` and the direct Live API; it does not require a map binding from `flourishcharts`.

Part B · Refresh the data

Run `flourish_map.R`. Each run does the following:

1. Pull fresh SPI data with `spiR` and shape it so the column names match the base bindings:

```
spi_regions <- spi_index() %>%
  filter(date == 2024) %>%
  transmute(Economy = iso3c, SPI.INDEX = SPI.INDEX) %>%
  filter(!is.na(Economy))
```

2. Point R at the base using its ID (styling is *not* re-created in R):

```
FLOURISH_VIS_ID <- "26427135"
```

3. Fetch the published base config at runtime and override only the data.

A projection map has four datasets (`regions`, `regions_geometry`, `points`, `lines`); the script swaps **only** `data.regions` with the fresh SPI values so the geometry and styling are preserved. Because any design change you make later in Flourish lives in the base, it is inherited automatically on the next run — **no R change needed**.

4. Write a self-contained HTML embed (`flourish_test.html`) and open it in the browser.

That HTML file is the deliverable that reflects the latest data.

To refresh later, just **re-run the script** — nothing else changes.

Part C · Where each responsibility lives

Task	Where you do it	How often
Design / restyle the chart	Flourish editor (the base)	Only when the look changes

Task	Where you do it	How often
Change which data is shown (year, indicator, filter)	R script (<code>spi_index()</code> + <code>filter/transmute</code>)	Whenever needed
Refresh with the newest SPI values	Re-run <code>flourish_map.R</code>	On demand / scheduled

Key mental model: the styled chart lives **once** in Flourish; R only feeds it data. You never copy styling into R, and you never paste data into Flourish by hand.

Quick refresh

```
Run flourish_map.R → new HTML is generated → upload/replace the HTML on the website
```

Running R updates the local HTML file. The chart on a website changes only after that new file is uploaded, unless deployment has been automated.

Why this is better than building visuals in R

Dimension	Option A: R visuals + publish	Option B: Flourish + R data (proposed)
Interactivity	Limited; hand-coded, heavy to maintain	Built-in (zoom, search, tooltips, animation)
Design & branding	Re-implemented in code per chart	Point-and-click in Flourish; reusable base
Maintenance	Ongoing R/JS/CSS upkeep per chart	Styling maintained in Flourish, not code
Who can edit the look	Only R developers	Any analyst via the Flourish editor
Refresh with new data	Re-run R export	Re-run R (data only)
Consistency across charts	Manual	One base → many charts inherit styling
Time to add a new chart	High (build from scratch)	Low (clone/adjust a Flourish template)

Bottom line: R does what R is best at (data), while Flourish does what it is best at (design and interactivity). The team maintains one visual design instead of rebuilding it in code for every update.

Before adopting it

- **The API does not overwrite the saved Flourish chart.** It renders a *live* chart from template + fresh data. The deliverable that updates is the R-generated HTML embed (or a hosted copy of it), not the chart sitting in the Flourish editor. This is a documented Flourish behavior, not a limitation of our implementation.
 - **Enterprise feature.** Flourish's Live API is an enterprise add-on and requires an API key. It would not work if we wanted to use a Flourish visualization for our package for everybody to use.
 - **API key handling.** The key is embedded client-side in the output. For public hosting we should restrict/rotate the key and avoid committing it to shared/synced folders.
 - **Automation for a public link.** Manual re-runs work today. For a always-fresh public URL, we host the generated HTML and schedule the R run (e.g., GitHub Actions + GitHub Pages).
-

References

- **Replicating a visualization** — the official source for this pipeline's core pattern: create & publish a styled *base* visualization, call the Live API with `base_visualisation_id`, override only `data/bindings`, and later restyle the base so every API chart inherits the change automatically.
- **Using the Flourish API in Python or R** — API key setup for R (`FLOURISH_API_KEY` in `~/Renvirom`, restart R) and the note that the Live API is an enterprise-level bolt-on.